ohmjs / ohm

Code | Issues 40 | Pull requests 5 | Discussions | Actions | Projects | Wiki

Watch 79

A library and language for building parsers, interpreters, compilers, etc.

MIT license

Contributing

5.5k stars | 225 forks | 79 watching | Branches | Activity | Custom properties | Tags

Public repository

39 Branches | 53 Tags | Go to file t | Go to file | Add file | Code

pdubroy last week

.github/workflows	Docker release gha (#588)	2 weeks ago
bin	chore: end support for Node 20...	2 weeks ago
doc	Docker release gha (#588)	2 weeks ago
docker	Docker release gha (#588)	2 weeks ago
examples	chore: end support for Node 20...	2 weeks ago
packages	compiler: detect excessively-de...	last week
.dockerignore	Dockerize (#578)	2 weeks ago
.gitignore	wasm: set up API reports with a...	last month
.mailmap	Update contributors.	9 years ago
.prettierignore	wasm: rename to 'compiler'	last month
.prettierrc	upgrade to ESLint 9	6 months ago
.release-please-manifest....	chore: release-please config	3 years ago
CONTRIBUTING.md	chore: Update repo references (...	3 years ago
LICENSE	chore: Update copyright date in...	4 years ago

README.md	Dockerize (#578)	2 weeks ago
biome.jsonc	wasm: rename to 'compiler'	last month
eslint.config.mjs	wasm: rename to 'compiler'	last month
package.json	Dockerize (#578)	2 weeks ago
pnpm-lock.yaml	chore: update pnpm lockfile	2 weeks ago
pnpm-workspace.yaml	rename recoverIterOrder -> rec...	3 months ago
tsconfig.json	BREAKING CHANGE: Change co...	5 years ago

README Contributing MIT license

Ohm • npm v17.5.0 Node.js CI passing chat on discord

Ohm is a parsing toolkit consisting of a library and a domain-specific language. You can use it to parse custom file formats or quickly build parsers, interpreters, and compilers for programming languages.

The *Ohm language* is based on parsing expression grammars (PEGs), which are a formal way of describing syntax, similar to regular expressions and context-free grammars. The *Ohm library* provides a JavaScript interface for creating parsers, interpreters, and more from the grammars you write.

- **Full support for left-recursive rules** means that you can define left-associative operators in a natural way.
- **Object-oriented grammar extension** makes it easy to extend an existing language with new syntax.
- **Modular semantic actions.** Unlike many similar tools, Ohm completely separates grammars from semantic actions. This separation improves modularity and extensibility, and makes both grammars and semantic actions easier to read and understand.
- **Online editor and visualizer.** The Ohm Editor provides instant feedback and an interactive visualization that makes the entire execution of the parser visible and tangible. It'll make you feel like you have superpowers. 💪

Some awesome things people have built using Ohm:

- Seymour, a live programming environment for the classroom.
- Shadama, a particle simulation language designed for high-school science.

- turtle.audio, an audio environment where simple text commands generate lines that can play music.
- A browser-based tool that turns written *Konnakkol* (a South Indian vocal percussion art) into audio.
- Wildcard, a browser extension that empowers anyone to modify websites to meet their own specific needs, uses Ohm for its spreadsheet formulas.

Getting Started

The easiest way to get started with Ohm is to use the interactive editor. Alternatively, you can play with one of the following examples on JSFiddle:

- Basic parsing example
- Arithmetic example with semantics

Resources

- Tutorial: Ohm: Parsing Made Easy
- The math example is extensively commented and is a good way to dive deeper.
- Examples
- Documentation
- For community support and discussion, join us on Discord, GitHub Discussions, or the ohm-discuss mailing list.
- For updates, follow on Bluesky or Mastodon.

Docker

You can use the Ohm CLI without a local Node.js installation via the Docker image:

```
docker run --rm -v $(pwd):/local ohmjs/ohm:latest compile my-grammar.ohm
```

For full usage instructions, including how to build the image locally and set up a development container, see doc/docker.md.

Installation

On a web page

To use Ohm in the browser, just add a single `<script>` tag to your page:

```
<!-- Development version of Ohm from unpkg.com -->
<script src="https://unpkg.com/ohm-js@17/dist/ohm.js"></script>
```

or

```
<!-- Minified version, for faster page loads -->
<script src="https://unpkg.com/ohm-js@17/dist/ohm.min.js"></script>
```

This creates a global variable named `ohm`.

Node.js

First, install the `ohm-js` package with your package manager:

- npm: `npm install ohm-js`
- Yarn: `yarn add ohm-js`
- pnpm: `pnpm add ohm-js`

Then, you can use `require` to use Ohm in a script:

```
const ohm = require('ohm-js');
```

Ohm can also be imported as an ES module:

```
import * as ohm from 'ohm-js';
```

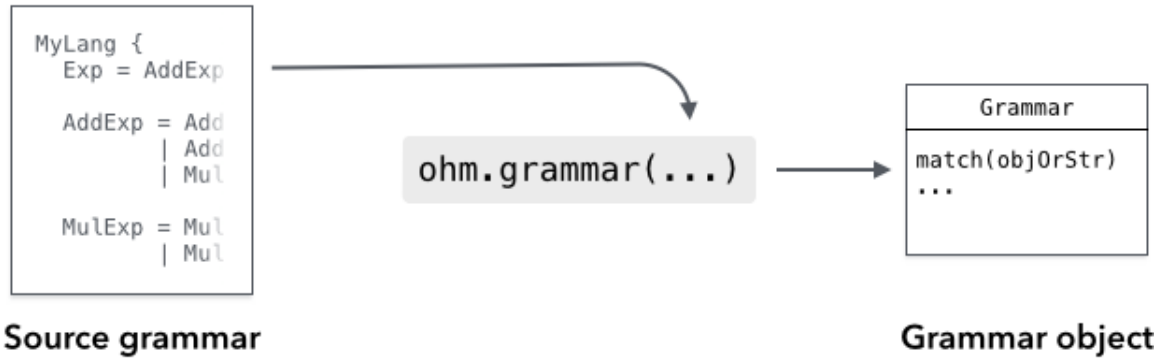
Deno

To use Ohm from Deno:

```
import * as ohm from 'https://unpkg.com/ohm-js@17';
```

Basics

Defining Grammars



To use Ohm, you need a grammar that is written in the Ohm language. The grammar provides a formal definition of the language or data format that you want to parse. There are a few different ways you can define an Ohm grammar:

- The simplest option is to define the grammar directly in a JavaScript string and instantiate it using `ohm.grammar()`. In most cases, you should use a template literal with String.raw:

```
const myGrammar = ohm.grammar(String.raw`
  MyGrammar {
    greeting = "Hello" | "Hola"
  }
`);
```

- **In Node.js**, you can define the grammar in a separate file, and read the file's contents and instantiate it using `ohm.grammar(contents)`:

  In `myGrammar.ohm`:

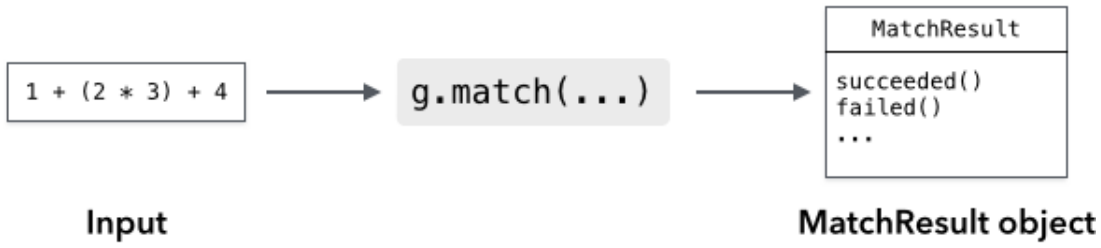
```
  MyGrammar {
    greeting = "Hello" | "Hola"
  }
```

  In JavaScript:

```
const fs = require('fs');
const ohm = require('ohm-js');
const contents = fs.readFileSync('myGrammar.ohm', 'utf-8');
const myGrammar = ohm.grammar(contents);
```

For more information, see Instantiating Grammars in the API reference.

Using Grammars



Once you've instantiated a grammar object, use the grammar's `match()` method to recognize input:

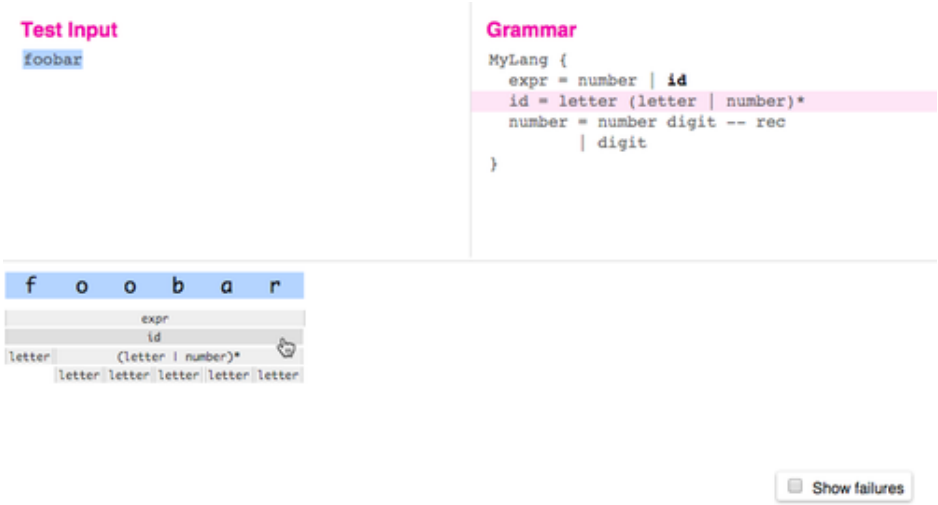
```
const userInput = 'Hello';
const m = myGrammar.match(userInput);
if (m.succeeded()) {
  console.log('Greetings, human.');
} else {
  console.log("That's not a greeting!");
}
```

The result is a MatchResult object. You can use the `succeeded()` and `failed()` methods to see whether the input was recognized or not.

For more information, see the [main documentation](#).

Debugging

Ohm has two tools to help you debug grammars: a text trace, and a graphical visualizer.



You can [try the visualizer online](#).

To see the text trace for a grammar `g`, just use the `g.trace()` method instead of `g.match`. It takes the same arguments, but instead of returning a MatchResult object, it returns a Trace object — calling its `toString` method returns a string describing all of the decisions the parser made when trying to match the input. For example, here is the result of `g.trace('ab').toString()` for the grammar `G { start = letter+ }`:

```
ab          ✓ start ⇒  "ab"
ab            ✓ letter+ ⇒  "ab"
ab              ✓ letter ⇒  "a"
ab                  ✓ lower ⇒  "a"
ab                    ✓ Unicode [Ll] character ⇒  "a"
b               ✓ letter ⇒  "b"
b                   ✓ lower ⇒  "b"
b                     ✓ Unicode [Ll] character ⇒  "b"
```

```
                ✗ letter
                    ✗ lower
                      ✗ Unicode [Ll] character
                    ✗ upper
                      ✗ Unicode [Lu] character
                    ✗ unicodeLtmo
                      ✗ Unicode [Ltmo] character
            ✓ end ⇒  ""
```

Publishing Grammars

If you've written an Ohm grammar that you'd like to share with others, see our suggestions for publishing grammars.

Contributing to Ohm

Releases 19

Ohm v17.5.0 Latest
on Feb 14

+ 18 releases

Packages

No packages published

Contributors 53

+ 39 contributors

Languages

JavaScript 49.6% TypeScript 27.9% Python 8.2% Liquid 6.8% HTML 4.9% Go 1.6%
Other 1.0%